COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 13, 2026	**DATE SUBMITTED:**	Oct 15, 2026
NAME:	Acelajado, John Vince D. Pangilinan Jr., Bienvenido B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12 [ACTIVITY NAME] Algorithmic Strategies

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Table 12-1. Algorithmic Strategies and Examples

Strategy	Algorithm	Analysis
Recursion	Fibonacci	Each term is by adding the two preceding values, and then it creates. a deeply branching call tree where identical subproblems are repetitively evaluated at every level of the descent.
Brute Force	Linear Search	The algorithm checks every single element in the dataset sequentially from start to finish until the target value is found or the entire list has been completely traversed.
Backtracking	Depth First Search (DFS)	The algorithm explores nodes by moving forward along a path whenever possible, and when no more unvisited nodes remain on the current path, it moves backwards to find and traverse alternative paths.
Greedy	Selection Sort	At each iteration, the algorithm makes a choice by scanning the remaining unsorted portion of the array to find the minimum element and then place it into its correct position.
Divide-And-Conquer	Merge Sort	This algorithm works by dividing

		the array into smaller parts into one element, and then conquering it by merging them together into the correct order.

Table 12-2. Memoization Implementation

Screenshot	**Code:**
	`#include <iostream>`
	`#include <vector>`
	`#include <algorithm>`
	`using namespace std;`
	
	`vector<int> memo;`
	`void memoAssign(int size, int value) {`
	`  memo.assign(size, value);`
	`}`
	
	`int getMinSteps(int n) {`
	`  if (n == 1) return 0;`
	`  if (memo[n] != -1) return memo[n];`
	`  int r = 1 + getMinSteps(n - 1);`
	`  if (n % 2 == 0) r = min(a: r, b: 1 + getMinSteps(n / 2));`
	`  if (n % 3 == 0) r = min(a: r, b: 1 + getMinSteps(n / 3));`
	`  memo[n] = r;`
	`  return r;`
	`}`
	
	`int main() {`
	`  int tests[] = {1, 4, 7, 10};`
	`  for (int n : tests) {`
	`  memoAssign(size: n + 1, value: -1);`
	`  cout << "n: " << n << " -> minimum steps: " << getMinSteps(n) << endl;`
	`  }`
	`  return 0;`
	`}`
	
	Output:
	`n: 1 -> minimum steps: 0`
	`n: 4 -> minimum steps: 2`
	`n: 7 -> minimum steps: 3`
	`n: 10 -> minimum steps: 3`

Analysis	**Acelajado** - The task here is to find the smallest number of steps required to get a given number to 1 using only the three types of operator it can be subtracted to 1, or divided by 2 or 3. Each of the steps it calculates the number of steps needed for each value is moved and uses min() to find the smallest count required. Then memoization works when it saves the past result if it has been stored even though one time it will make the program optimize and run faster. **Pangilinan** - The code finds the fewest steps needed to turn a number into 1. It starts with the big number and works its way down to smaller numbers. For each number, it tries three moves: subtract 1, divide by 2 and divide by 3. It counts the steps for each move and keeps the one with the fewest steps. After it solves a number, If the same number comes up again, it just reads the saved answer instead of solving it again, so the output would be, 1 needs 0 steps, 4 needs 2 steps, 7 needs 3 steps, and 10 needs 3 steps.

Table 12-3. Bottom-Up Dynamic Programming Implementation

Screenshot	**Code:**
	``cpp
	#include <iostream>
	#include <algorithm>
	using namespace std;
	
	int getMinSteps(int n)
	{
	    int dp[n + 1], i;
	    dp[1] = 0;
	    for (i = 2; i <= n; i++)
	    {
	        dp[i] = 1 + dp[i - 1];
	        if (i % 2 == 0) dp[i] = min(dp[i], 1 + dp[i / 2]);
	        if (i % 3 == 0) dp[i] = min(dp[i], 1 + dp[i / 3]);
	    }
	    return dp[n];
	}
	
	int main()
	{
	    int numbers[] = {1, 3, 4, 6};
	    for (int i = 0; i < 4; i++)
	    {
	        cout << "n: " << numbers[i] << " -> minimum steps: "
	             << getMinSteps(numbers[i]) << endl;
	    }
	    return 0;
	}
	``

	Output:
	`n: 1 -> minimum steps: 0`
	`n: 3 -> minimum steps: 1`
	`n: 4 -> minimum steps: 2`
	`n: 6 -> minimum steps: 2`

Analysis	**Acelajado** - The code computes the minimum number of steps required to decrease a number to 1 by using bottom up strategy. It begins at 1 then it will calculate its step 0. It then moves up through each number one by one and sees if subtracting 1 or dividing by 2 or 3 requires the least number of steps. It checks whether the division is allowed by using the module, reuses the answers which it has stored in the dp list, and outputs the answer for each test case. **Pangilinan** - The code finds the fewest steps to turn a number into 1. Unlike the last code, it starts from the small numbers and works up to the big one, after that there is a maker of list called dp. The list is the one who keeps the answer for every number, now it starts at 1 and sets its answer to 0 because it is already 1 and needs no step and then it solves the numbers one by one, from the smallest to the biggest. For each number, it looks at the moves it can use and checks which move leads to the fewest steps. It does this by using the answers of the smaller numbers that were already saved, and it keeps the best one.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Pseudocode (+ Solving Problem by Hand)

Pseudocode

```
START
   Create a 4 x 4 matrix
   Set the target cost to 25

   FUNCTION countPaths(row, col, cost)
      IF row or column is outside the matrix OR cost < 0
         RETURN 0
      END IF

      Subtract the current cell value from cost

      IF the current cell is the last cell
         IF cost = 0
            RETURN 1
         ELSE
            RETURN 0
         END IF
      END IF

      RETURN countPaths(row + 1, col, cost)
          + countPaths(row, col + 1, cost)
   END FUNCTION

   Display countPaths(0, 0, target cost)
END
```

Solving Problem by Hand

- Our target cost is 25.
- We need to start at the top left corner of the matrix and end at the bottom right corner of the matrix.
- We can only move by right or down then adding each visited cell value together to compute the total.
- Find the paths that can reach 25.
- For Example: 4+6+7+8+1+7+3 = 36
- Based on the possible paths that we test we can't get the target cost 25.

C++ Code

Code:

```cpp
#include <iostream>
using namespace std;

int matrix[4][4] = {
    {4, 7, 1, 6},
    {6, 7, 3, 9},
    {3, 8, 1, 2},
    {7, 1, 7, 3}
};

int countPaths(int row, int col, int cost){
    if (row >= 4 || col >= 4 || cost < 0) return 0;
    cost -= matrix[row][col];
    if (row == 3 && col == 3){
        if (cost == 0) return 1;
        else return 0;
    }
    return countPaths(row + 1, col, cost) + countPaths(row, col + 1, cost);
}

int main()
{
    int cost = 25;
    cout << "Number of paths with cost 25: "
         << countPaths(0, 0, cost) << endl;
    return 0;
}
```

Output:

```
Number of paths with cost 25: 0
```

Analysis of the Code

Acelajado

- In this supplementary code my analysis is the this uses a recursive backtracking where it test every path from the top left to the bottom right corner of the 4x4 matrix, what happens is it subtracts each cell value from that starting cost of 25 also while only the it is moving right or down and will stop if the path goes out of bound or will exceeds the cost. Since none of the routes get the target cost 25 the output will display 0 paths.

Pangilinan

- The code checks every possible path from the top-left corner of the 4 x 4 matrix to the bottom-right corner. It can only move right or down, and it counts the paths whose numbers add up to exactly 25 and At each cell, it subtracts the cell's number from the cost. If the path goes outside the matrix or the cost goes below 0, it stops and returns 0. For every other cell, it adds the results of moving down and moving right, so it ends up counting all the good paths.

Conclusion
Acelajado - My conclusion in this activity is that I learned the different algorithmic approach methods where it includes the recursion, brute force, backtracking, greedy choice, divide and conquer and also the dynamic programming. This different algorithm is used for different purposes and each one of them has advantages and disadvantages so when making a project you should determine what type of algorithm was best to use. In the first half of the lab where we code the bottomup dynamic programming and the topdown memoization techniques this showed me how storing past results will prevent unnecessary calculations and will optimize the algorithm which makes it faster. Then on supplementary activity it is about exploring the 4x4 matrix recursively and calculates all the possible paths to the target while observing how the path space is pruned for invalid paths and how certain the target conditions are evaluated. **Pangilinan** - In this activity, I learned how different algorithms solve problems in such different ways and . Each strategy has its own way of breaking a problem down, and choosing the right one makes the solution faster and easier to understand. After analyzing the code, I saw that the same kind of problem can be solved in more than one way, some ways are better than others.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar Correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Only use for grammar correction

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.